



ESTRUTURA DE DADOS

Dra. Maíla de Lima Claro
maila.claro@ifpi.edu.br



Introdução à Linguagem C

Exemplo de um programa em C

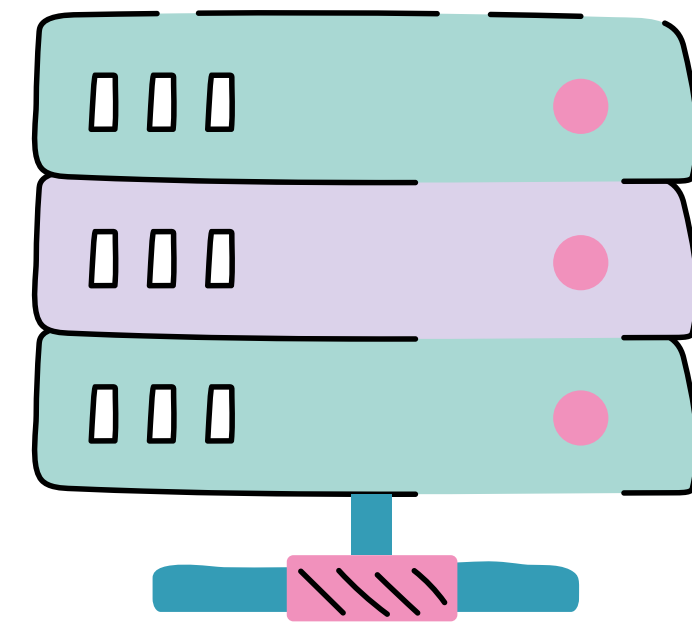
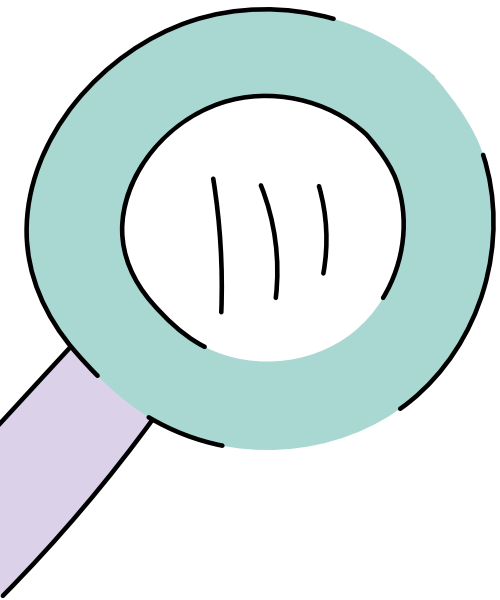
```
// Exemplo de programa em C
// Isto é uma linha de comentário
void main()
{
    int a;           // declara a variável "a"
    a = 3 + 2;       // soma 3 com 2
}
```

- Um programa em C é composto por um conjunto de **Funções**. A função pela qual o programa começa a ser executado chama-se **main**.
- Após cada comando em C deve-se colocar um ; (**ponto-e-vírgula**).
- Um programa em C deve ser **Indentado** para que possa ser lido com mais facilidade.

Introdução à Linguagem C

IDENTIFICADORES

- São os nomes que podem ser dados para variáveis e funções.
- Para a escolha destes nomes é necessário seguir algumas regras:
 - Um Identificador deve iniciar por um letra ou um “_” (Underscore)
 - Deve-se usar nomes significativos dentro do contexto do programa;
 - C é uma linguagem *case-sensitive*, ou seja, faz diferença entre nomes com letras maiúsculas e nomes com letras minúsculas. Peso e peso são diferentes;
 - Costuma-se usar maiúsculas e minúsculas para separar palavras: "PesoDoCarro";
 - Deve ser diferente dos comandos da linguagem;
 - Deve ter no máximo 31 caracteres;
 - Pode conter números a partir do segundo caracter;
 - Exemplos:
 - Idade, Contador, PesoDoCarro,
 - Usuario_1, CorDaPagina, RaioDoCirculo



Variáveis

- Uma variável é uma posição de memória que pode ser identificada através de um nome.
- Podem ter seu conteúdo alterado por um **comando de atribuição**.
- Após a atribuição mudam de valor.

```
int a,b, SomaGeral;  
  
a = 3; // a recebe o valor 3  
  
b = a * 2; // b recebe o dobro do valor de a  
  
c = a + b + 2; // c recebe 11
```



Tipos de Variáveis

- Todas as variáveis em C tem um **tipo**;
- Cada tipo define os valores que a variável pode armazenar;
- Cada tipo ocupa uma certa quantidade de memória.

Tipo	Valores Válidos
char	Letras e símbolos `a`, `b`, `h`, `^`, `*`, `l`, `0`
int	De -32767 até 32767 (apenas números inteiros)
float	De -3.4×10^{38} até $3.4 \times 10^{+38}$ com até 6 dígitos de precisão
double	De -1.7×10^{308} até $1.7 \times 10^{+308}$ com até 10 dígitos de precisão



Declaração de Variáveis

- Todas as variáveis **tem que ser declaradas antes** de serem usadas;
- Não há uma inicialização implícita na declaração.

```
// Exemplo de programa em C

#include <stdio.h> // Arquivo de cabeçalho (header)
void main()
{
    int contador;           // declarações simples
    float PrecoDoQuilo;
    double TaxaDeCambio;
    char LetraDigitada;
    int IdadeManoel, IdadeJoao, IdadeMaria; // Pode colocar mais de uma variável na
                                           // na mesma linha

    double TaxaDoDolar,
           TaxaDoMarco,
           TaxaDoPeso,      // Também pode trocar de linha no meio
           TaxaDoFranco;

    .....
}
```



Inicialização de Variáveis na Declaração

```
// Exemplo de programa em C

#include <stdio.h> // Arquivo de cabeçalho (header)
void main()
{
    int NroDeHoras = 0;           // declara e inicializa com Zero
    float PrecoDoQuilo = 10.53;  // declara e inicializa com 10.53
    double TaxaDoDolar = 1.8,
           TaxaDoMarco = 1.956,
           TaxaDoPeso = 1.75,
           TaxaDoFranco = 0.2;

    .....
}
```



Constantes

- Constantes são identificadores que não podem ter seus valores alterados durante a execução do programa.
- Para criar uma constante existe o comando `#define` que, em geral é colocado no início do programa-fonte.

Exemplos

```
#define LARGURA_MAXIMA 50           // Não se coloca ponto-e-vírgula após o valor
#define NRO_DE_DIAS_DA_SEMANA 7
#define NRO_DE_HORAS_DO_DIA 24
#define VALOR_DE_PI 3.1415

void main ()
{
    int TotalDeHoras;

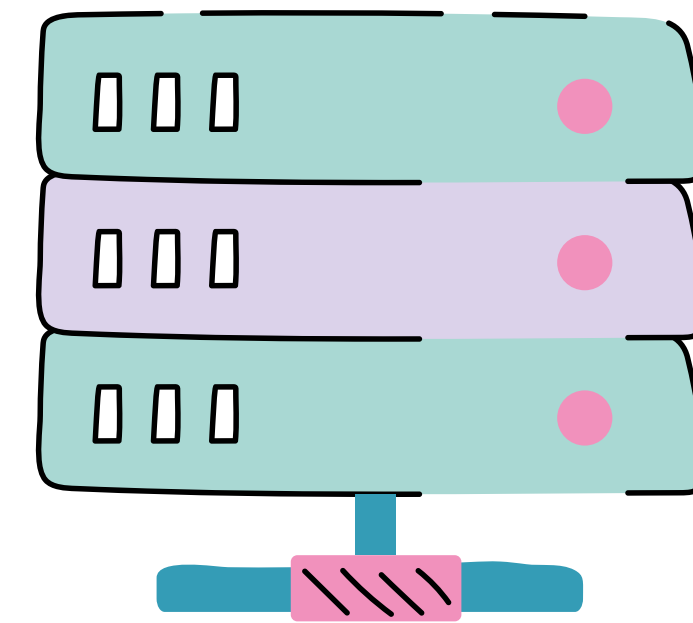
    TotalDeHoras = 10 * NRO_DE_DIAS_DA_SEMANA * NRO_DE_HORAS_DO_DIA;
    .....
}
```



Operadores Aritméticos

Operação	Operador Aritmético	Exemplo	Exemplo em C/C++
Adição	+	$f + 7$	$f + 7$
Subtração	-	$p - c$	$p - c$
Multiplicação	*	bm ou $B \times m$	$b * m$
Divisão	/	x/y ou $x \div y$ ou $\frac{x}{y}$	x / y
Módulo	%	$r \bmod s$	$r \% s$

- Observações:
 - Operador módulo %: resulta no resto da divisão inteira (somente usado com operandos inteiros)
 - Exemplo: $7 \% 4$ é igual a 3



Regras da Precedência de Operadores

Prioridade 1

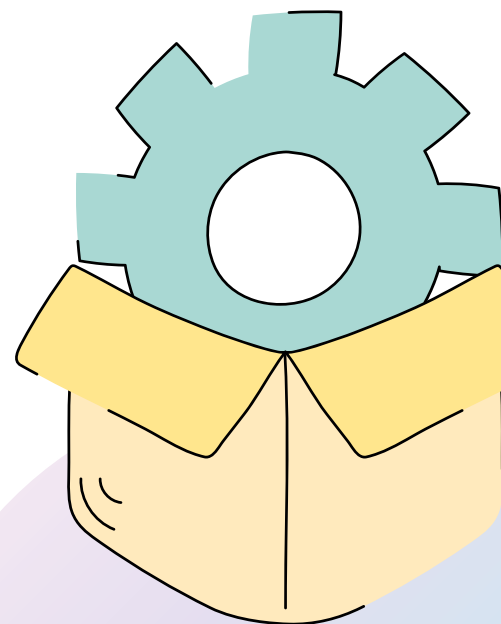
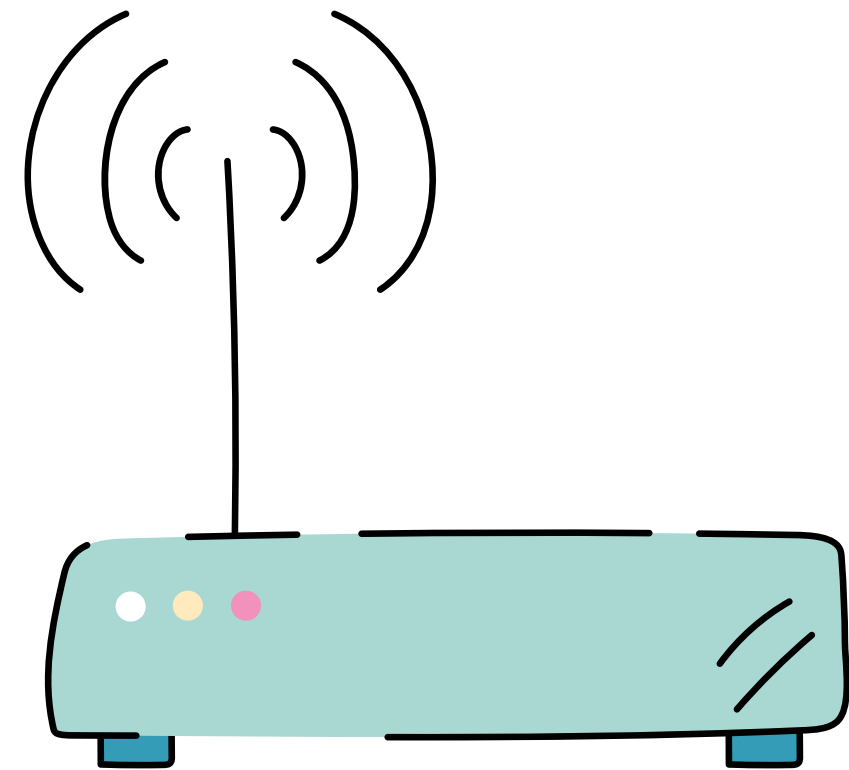
Operadores entre parênteses são avaliados primeiro; note que o parênteses quebra a precedência de um operador.

Prioridade 2

A seguir, aplicam-se as operações de **multiplicação**, **divisão** e **módulo**. Se uma expressão contém vários desses operadores, as operações são aplicadas da esquerda para a direita.

Prioridade 3

Por último aplicam-se a **adição** e a **subtração**. Se há vários + e -, a aplicação ocorre da esquerda para a direita.

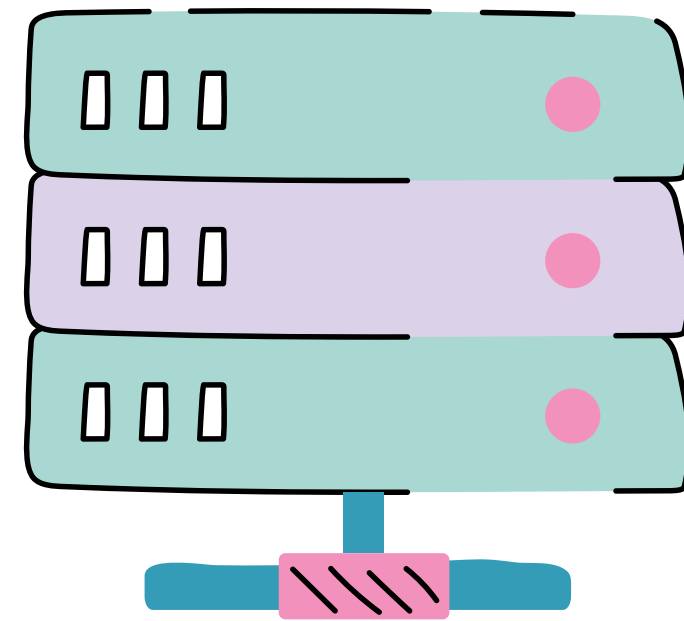


Comandos de Saída

- A função **printf** é parte da biblioteca `<stdio.h>`:
 - Utilizada para imprimir na tela.
 - Exemplo de uso:

```
printf("Olá mundo!!!\n");
```

Mas.. Como imprimir um número inteiro?

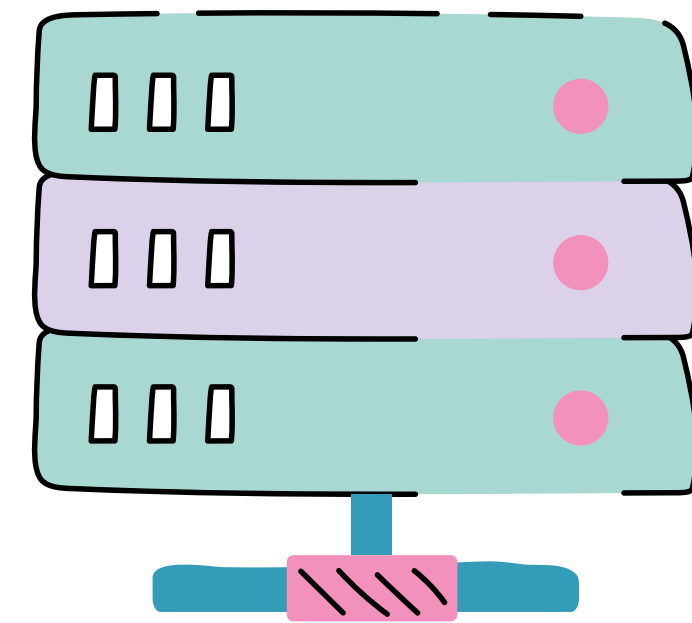


A Função printf()

- Será assim:
 - `printf(10);`
- Ou assim:
 - `int valor = 10;`
 - `printf(valor);`



Os códigos acima produzirão um **erro**, pois `printf` deve receber um texto/formato (entre aspas), não um inteiro (seja valor ou variável).

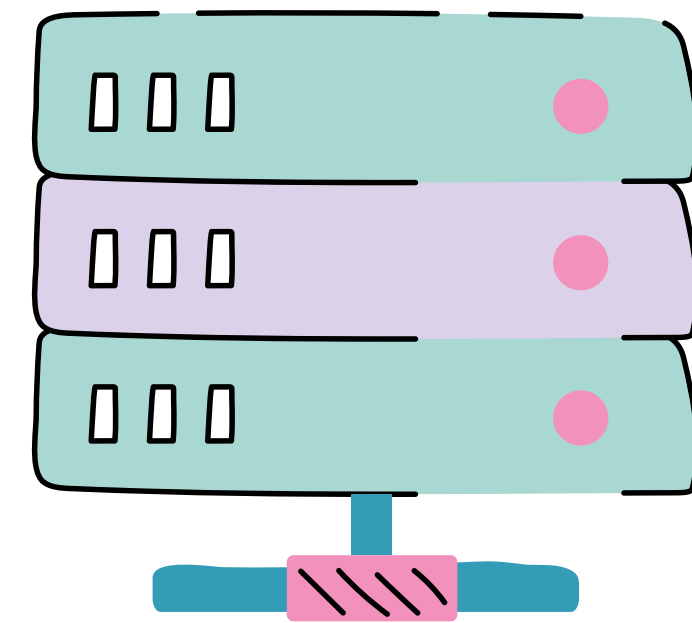
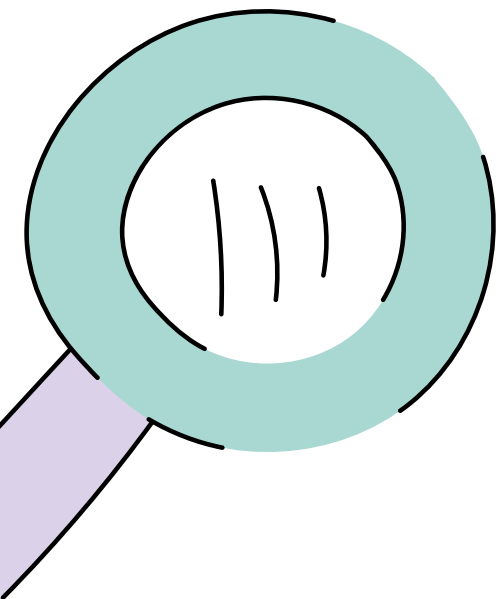


A Função printf()

- Uso de `printf`:
 - `printf(formato, valor/variável);`
- Exemplo:

```
printf("%d", 10);
```
- (note que `"%d"` é usado para números inteiros)

- Alguns possíveis formatos para o comando `printf`:
 - `"%d"`: `int` (número inteiro)
 - `"%ld"`: `long long` (número inteiro)
 - `"%f"`: `float` (ponto flutuante)
 - `"%lf"`: `double` (ponto flutuante)
 - `"%c"`: `char` (caractere)
 - `"%s"`: `string` (cadeia de caracteres)



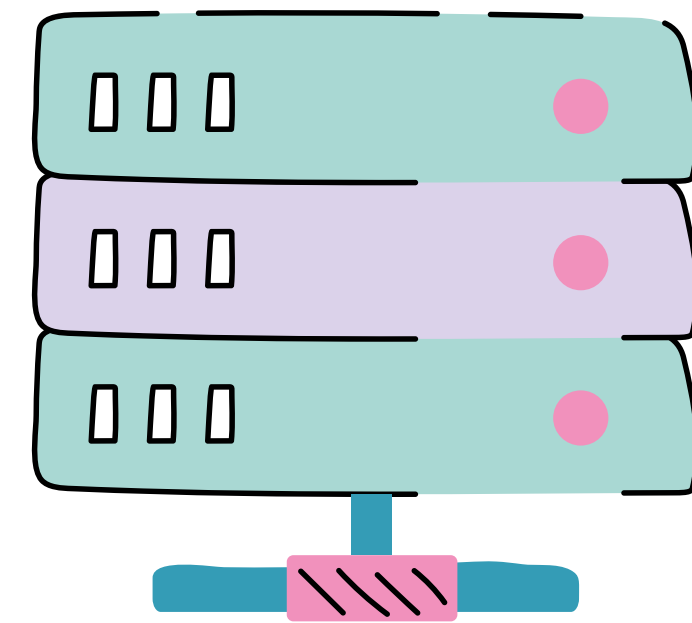
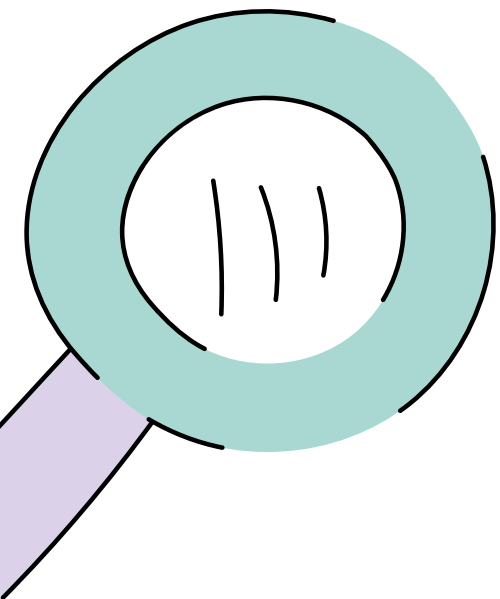
A Função printf()

- Note que é possível mesclar formato com texto, como por exemplo em "O resultado é %d"

```
int n1 = 10;  
int n2 = 20;  
int soma = n1 + n2;  
printf("A soma de %d e %d é igual a %d.\n", n1, n2, soma);
```

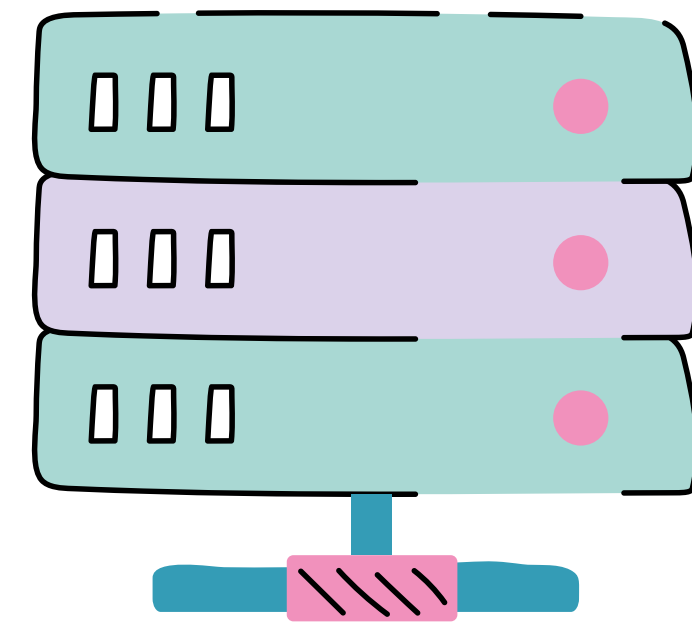
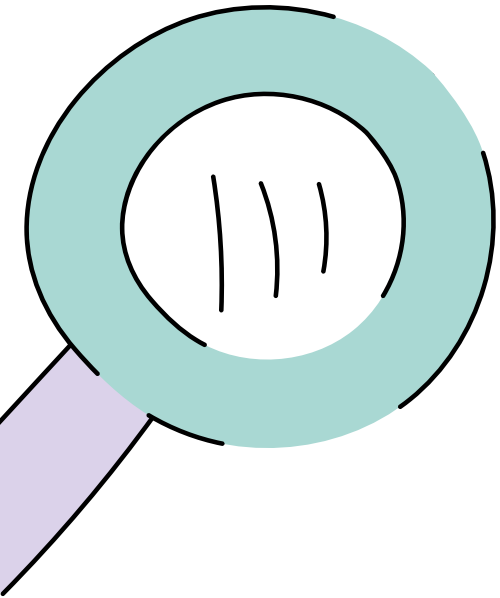
- Naturalmente, o código acima produzirá a saída:

```
A soma de 10 e 20 é igual a 30.
```



Formatando a Saída

- A função `printf` permite formatar a saída de dados. O usuário pode especificar, entre outros:
 - número de casas decimais;
 - número de caracteres ocupados pela impressão.
- Exemplos:
 - `"%3d"`: um `int` usando no mínimo 3 espaços
 - `"%-3d"`: um `int` usando no mínimo 3 espaços (alinhado à esquerda)
 - `"%5s"`: uma `string` usando no mínimo 5 espaços
 - `"%.3f"`: um `float` usando 3 casas decimais
 - `"%3f"`: um `float` usando no mínimo 3 espaços
 - `"%5.3f"`: um `float` usando 3 casas decimais e no mínimo 5 espaços



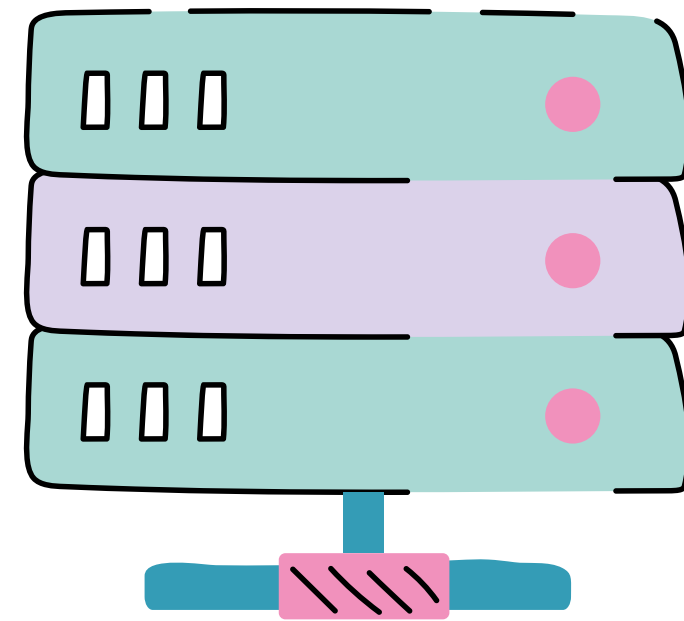
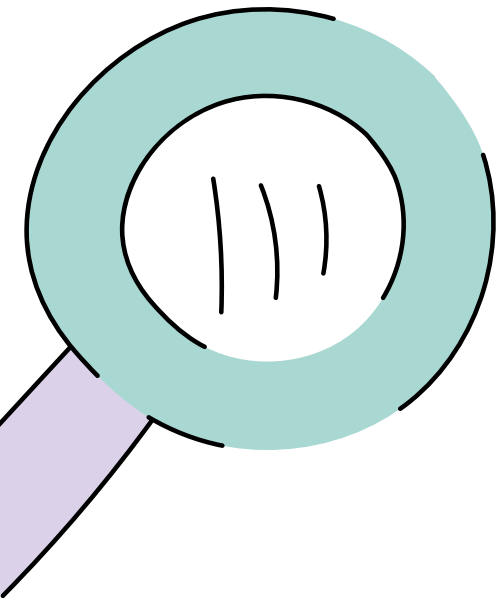
Formatando a Saída

- Exemplo:

```
printf("%-3s %8s\n", "Var", "Val");  
printf("%-3s %8.1f\n", "x", 10.222);  
printf("%-3s %8.1f\n", "y", 20.33);  
printf("%-3s %8.1f\n", "z", 30);
```

- Imprimirá na saída:

Var	Val
x	10.2
y	20.3
z	30.0



Formatando a Saída

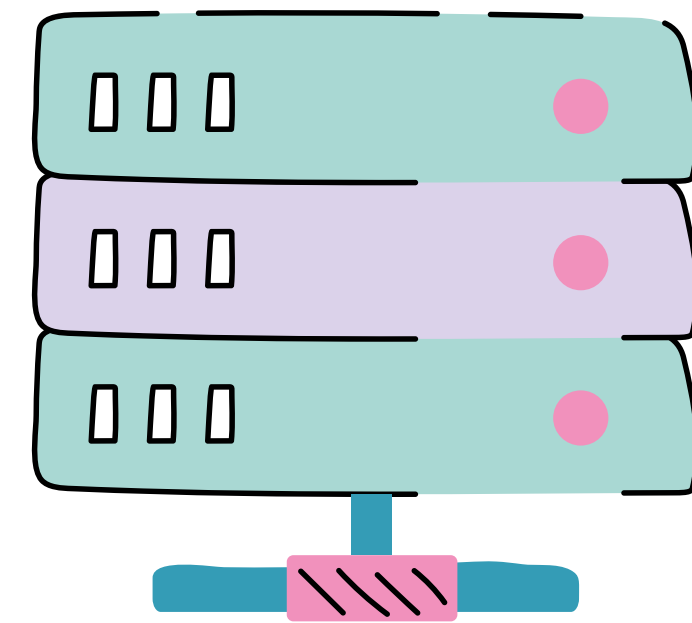
- Caracteres especiais:
 - `\n`: quebra de linha, ou seja, passa para a linha debaixo;
 - `\t`: tabulação horizontal, equivalente a um **tab**;
 - `\"`: aspas duplas;
 - `\'`: aspas simples ou apóstrofo;
 - `\\`: barra invertida
 - `\a`: ???**beep** ;)

Exemplo

```
printf( "Var \t Val\n");  
printf( "x  \t 10\n");  
printf( "y  \t 20\n");  
printf( "z  \t 30\n");
```

Resultado

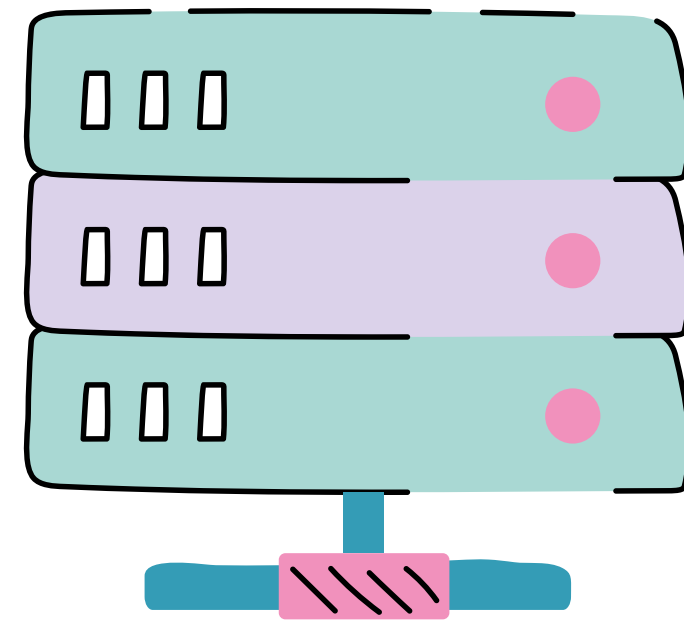
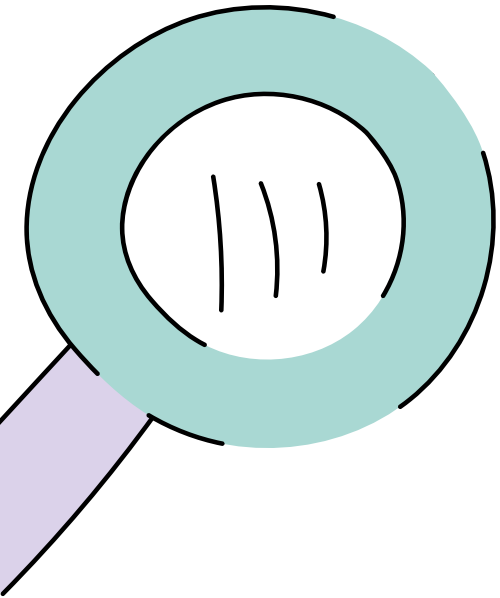
Var	Val
x	10
y	20
z	30



Comandos de Entrada

- A função `scanf` também é parte da biblioteca `<stdio.h>`:
 - Utilizada para ler da entrada padrão (terminal).
 - O `scanf` tem algumas (grandes) diferenças em relação ao `printf`:
 - A função `printf` imprime texto e o **valor** de variáveis.
 - A função `scanf` **altera o conteúdo** das variáveis.
 - Alterar conteúdo equivale a **modificar o que está na memória**.
-
- Uso de `scanf`:
 - `scanf(formato, endereços de memória);`
 - Exemplo:

```
int x;  
scanf("%d", &x);
```
 - (note que `"%d"` é usado para números inteiros)



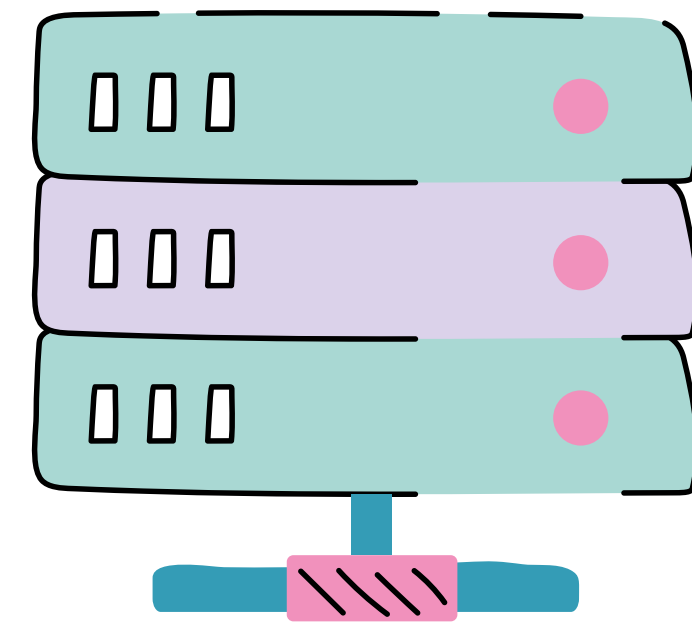
Função Scanf()

- Porquê os códigos abaixo geram **erros**?

```
int x;  
scanf(x);
```

```
double valor = 10.0;  
scanf(valor);
```

- `scanf` deve receber um texto/formato (entre aspas), não um `int` ou `double` (seja valor ou variável).
- `scanf` deve receber um endereço de memória, e não um valor.



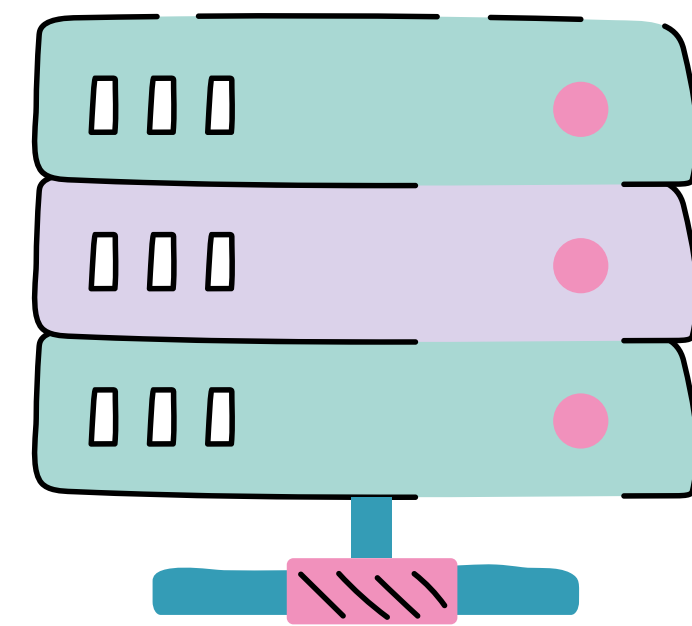
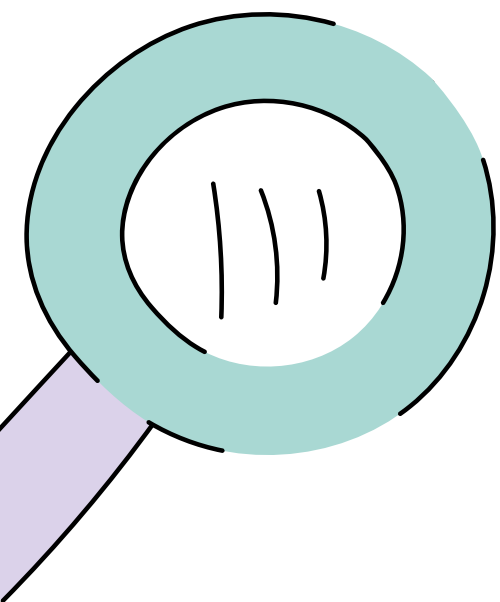
Função Scanf()

- E os códigos a seguir? Também geram **erros**?

```
int x;  
scanf("%d", x);
```

```
double valor = 10.0;  
scanf("%lf", valor);
```

- Sim: `scanf` deve receber **endereços de memória**, não valores.



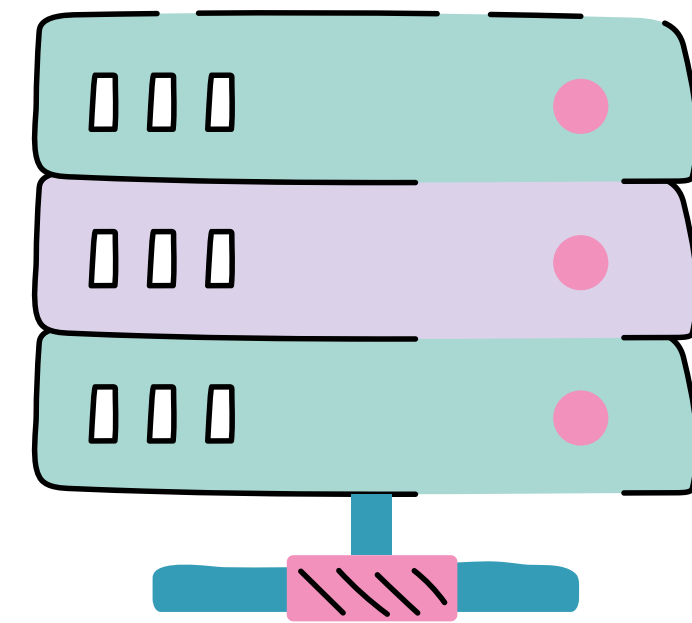
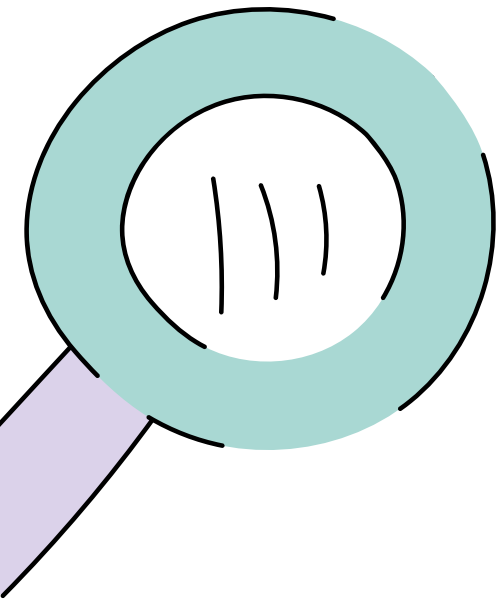
Função Scanf()

- Assim, para ler da entrada padrão usamos um “formato” e indicamos o endereço de memória como próximo argumento.
- Exemplos:

```
int x;  
scanf("%d", &x); // lê um inteiro da entrada padrão
```

```
char c;  
scanf("%c", &c); // lê um caractere da entrada padrão
```

```
int n1, n2, soma;  
scanf("%d %d", &n1, &n2); // lê dois inteiros da entrada padrão  
soma = n1 + n2;  
printf("A soma de %d e %d eh igual a %d", n1, n2, soma);
```



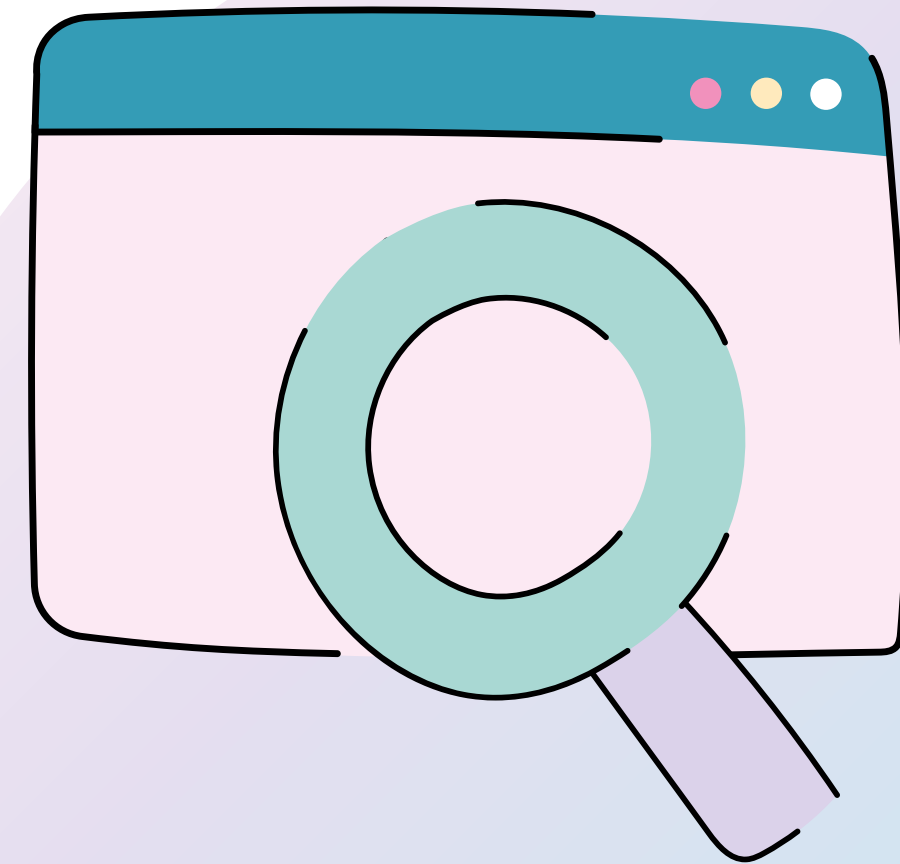
Exemplos

- 1 Elabore um programa em C que lê o valor das variáveis x , y e z do tipo `int`. Em seguida, calcule o resultado da expressão a seguir:

$$r = x^3 + y^2 + xyz$$

Imprima o resultado no formato do exemplo abaixo:

```
x = 10, y = 10, z = 10  
r = 2100
```

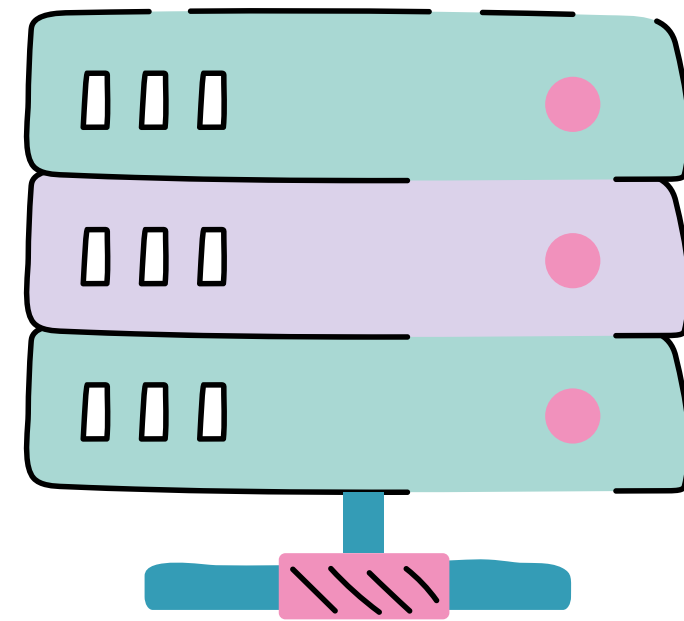
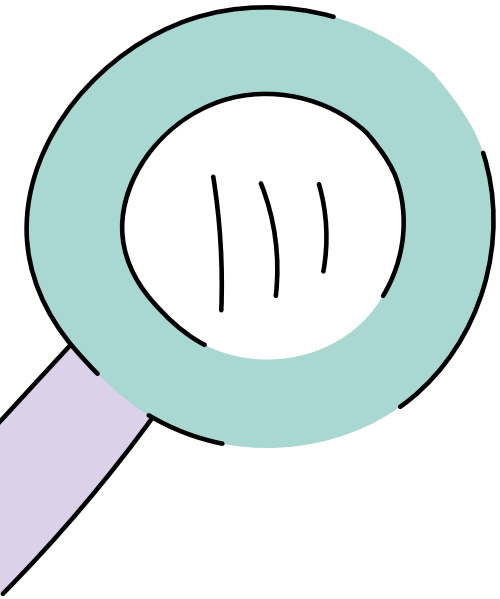


COMANDOS EM C



Comando if/else

- A tomada de decisão é um dos procedimentos mais comuns em programas de computadores, pois é através deles que o computador decide que procedimento executar.
- A linguagem C nos permite utilizar três comandos de decisão:
 - if
 - if-else
 - switch



Comando if/else

Exemplo 1: Aluno: Aprovado, Prova Final ou Reprovado.

```
#include <stdio.h>
// Função principal. Inicia a execução do programa
int main(void)
{
    int nota;
    printf("Digite sua nota");
    scanf("%d", &nota);
    if(nota>=7)
        printf("Aprovado");
    else if(nota<7 && nota >=4)
        printf("Prova Final");
    else
        printf("Reprovado");
    return 0; // indica que o programa terminou com sucesso
}
```

Comando if/else

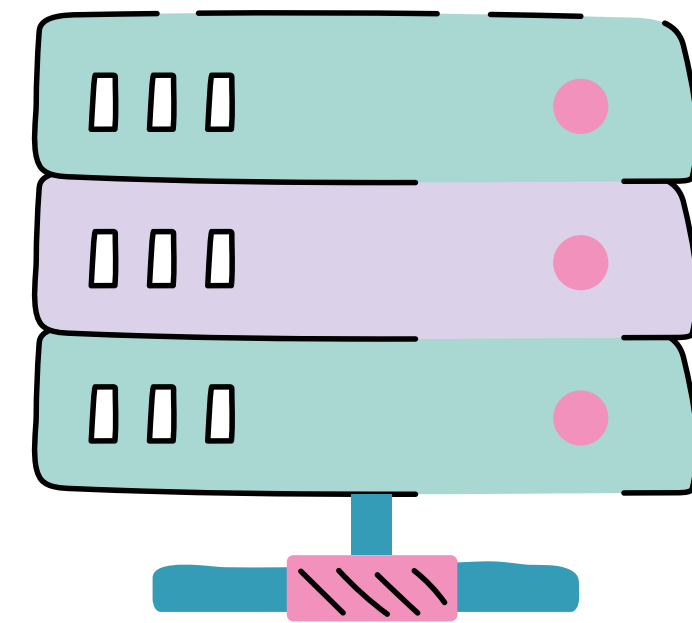
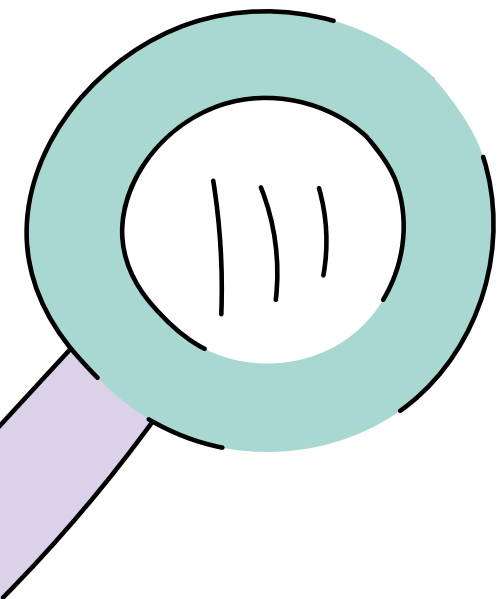
Exemplo 2: Escreva um programa que leia as medidas dos lados de um triângulo e escreva se ele é Equilátero, Isósceles ou Escaleno. Sendo que:

Triângulo Equilátero possui os 3 lados iguais;

Triângulo Isóscele: possui 2 lados iguais.

Triângulo Escaleno: possui 3 lados diferentes

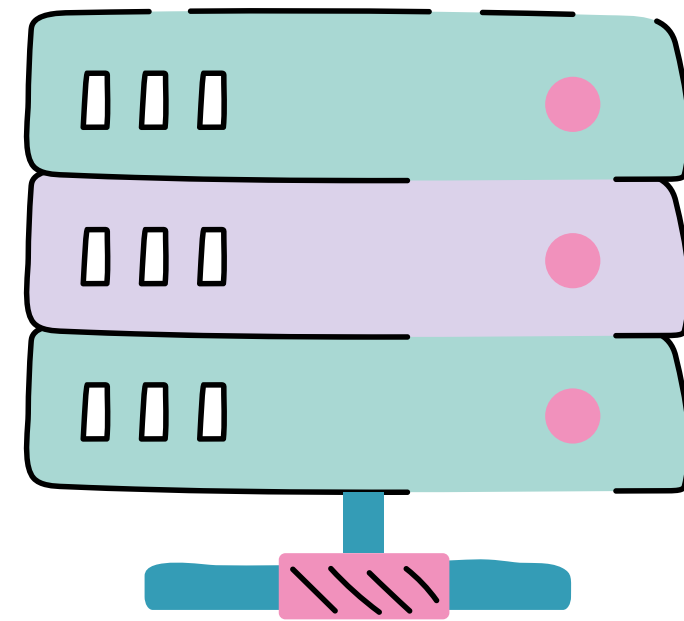
Exemplo 3: Escreva um programa que leia três valores inteiros e informe qual o maior valor entre eles.



Comando Switch

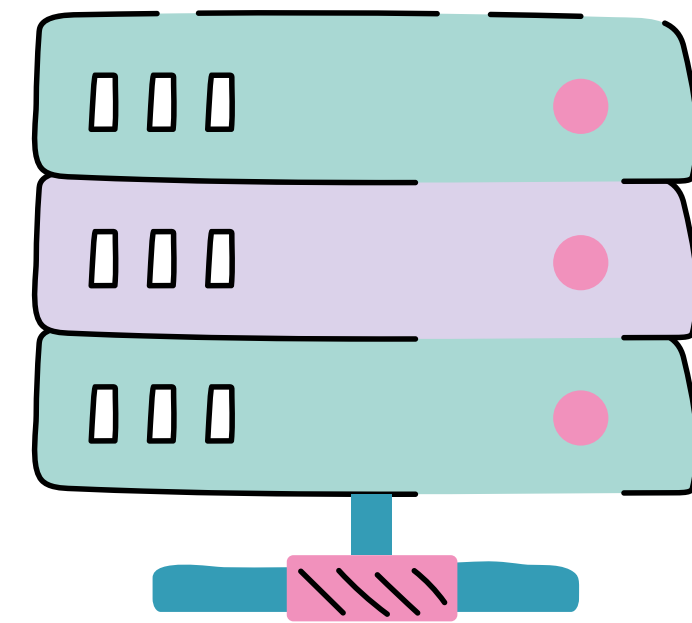
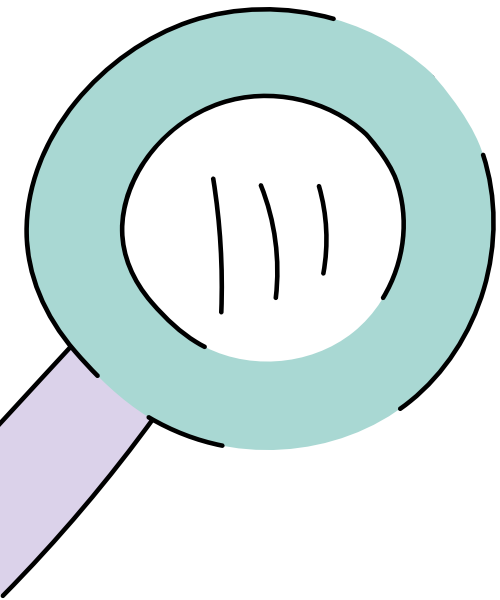
- Este comando nos permite selecionar uma opção entre várias alternativas. A variável será validada e conforme seu valor executará a opção na qual se enquadra.
- Um exemplo bem simples é apresentado a seguir sobre um programa que informa se a pessoa é mais adulto ou velho.

```
#include <stdio.h>
#include <stdlib.h>
// Função principal. Inicia a execução do programa
int main()
{
    int i; char sn;
    printf("Você tem mais de 70 anos? (S/N)");
    scanf("%c", &sn);
    switch(sn)
    {
        case 's': printf("Você está velho! \n"); break;
        case 'S': printf("Você está velho! \n"); break;
        case 'n': printf("Você está Adulto! \n"); break;
        case 'N': printf("Você está Adulto! \n"); break;
    }
    return 0; // indica que o programa terminou com sucesso
}
```



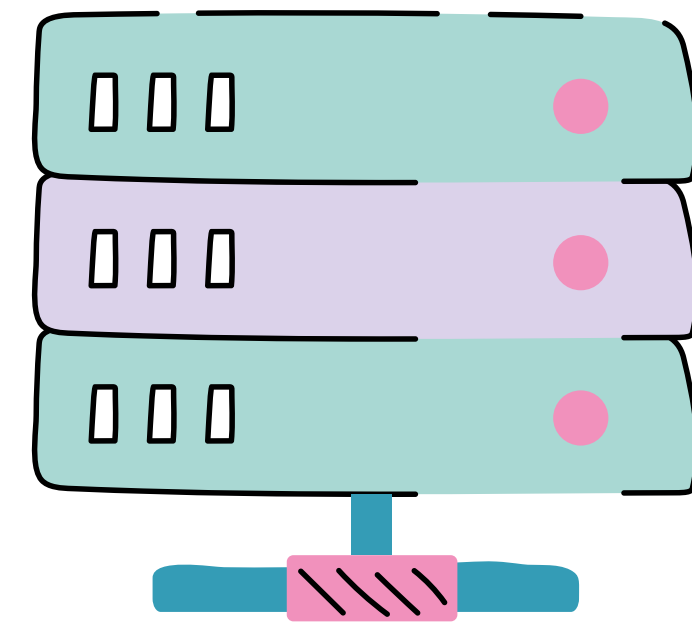
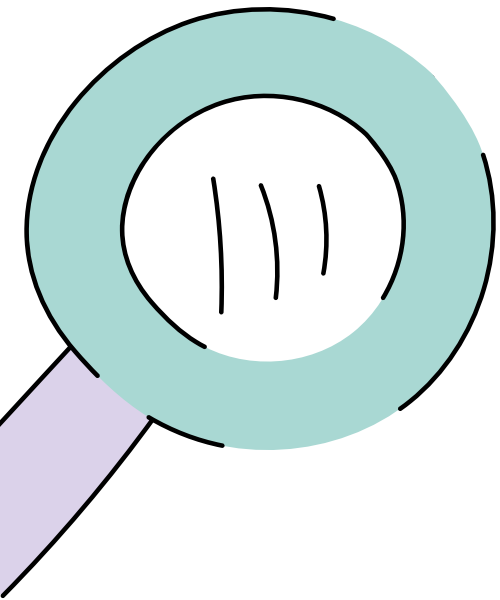
Comando Switch

- **Exemplo 4:** Escreva um programa em C que pede a idade da pessoa e deve informar se ela é bebê, criança, adolescente, adulto ou idoso.
 - 0 até 4 anos é bebê;
 - 5 até 12 anos é criança;
 - 13 até 21 anos é adolescente;
 - 22 até 60 anos é adulto;
 - Acima de 60 anos é idoso.



Comando Switch

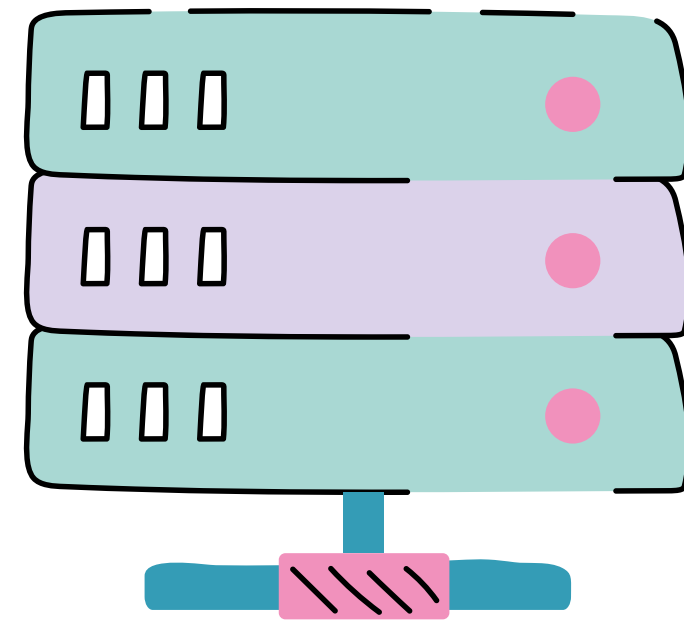
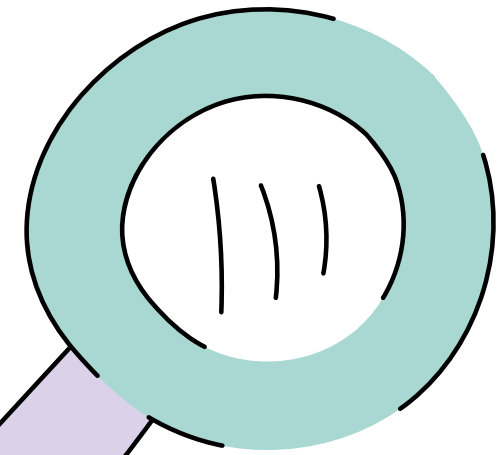
- **Exemplo 5:** Calculadora: peça para o usuário digitar a operação que deseja fazer e depois dois números e forneça o resultado da operação. (Use switch)
- **Exemplo 6:** Desenvolva um programa para verificar o dia da semana a fim de exemplificar a utilização do switch...case. O usuário vai digitar um número de 1 a 7 e o programa vai retornar o dia da semana equivalente ao número.



Comando For

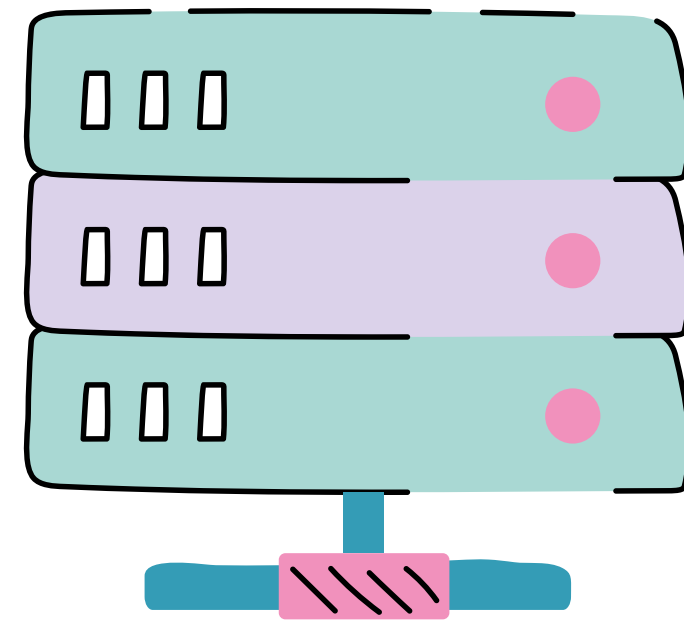
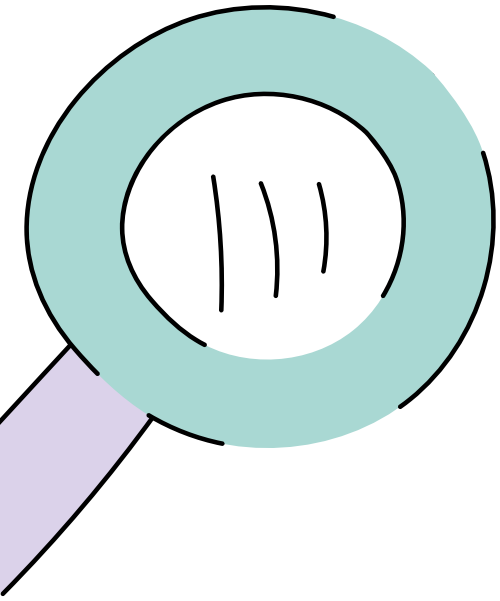
Exemplo 7: Elaborar um programa que imprima os número de 1 a 10.

```
#include <stdio.h>
// Função principal. Inicia a execução do programa
int main()
{
    int cont;
    for(cont=0;cont<=10;cont++)
    {
        printf("\n%d", cont);
    }
    return 0; // indica que o programa terminou com sucesso
}
```



Comando For

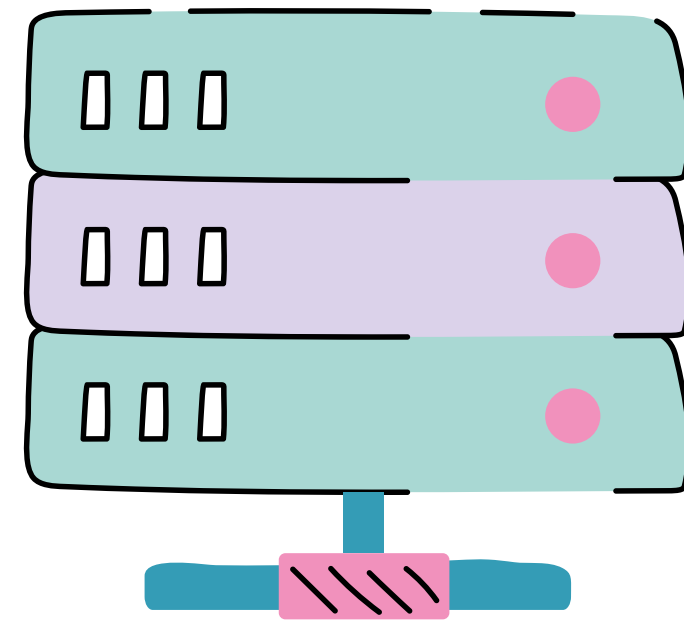
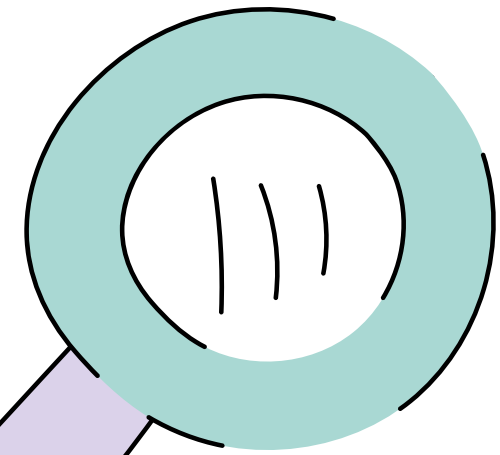
- **Exemplo 8:** Ler um valor inteiro (aceitar somente valores entre 1 e 10) e escrever a tabuada de 1 a 10 do valor lido.
- **Exemplo 9:** Fazer um programa para encontrar todos os números pares entre 1 e 100.
- **Exemplo 10:** Ler 10 números e imprimir quantos são pares e quantos são ímpares.
- **Exemplo 11:** Leia várias idades e calcule a média entre as idades (usar uma variável para idade).



Comando While

Exemplo 12: Mesmo programa do for, só que usando while na contagem de 1 até 10.

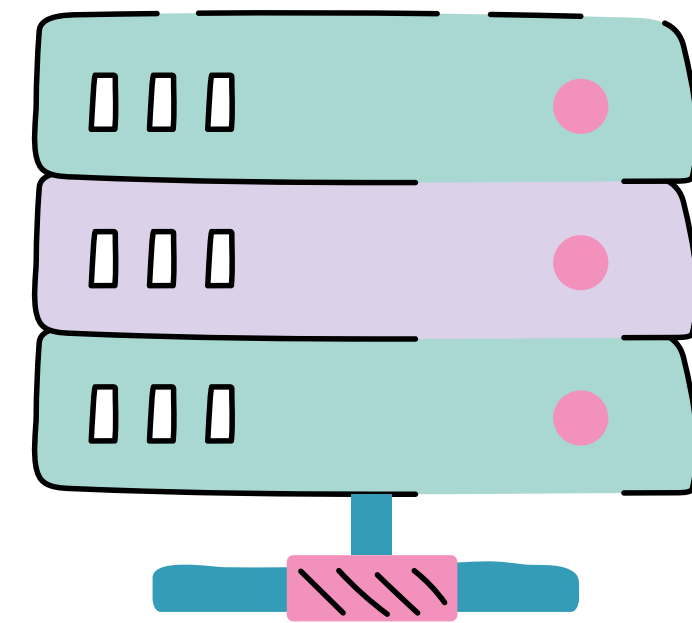
```
#include <stdio.h>
// Função principal. Inicia a execução do programa
int main()
{
    int cont=0;
    while(cont<=10)
    {
        printf("\n%d", cont);
        cont++;
    }
    return 0; // indica que o programa terminou com sucesso
}
```



Comando Do/While

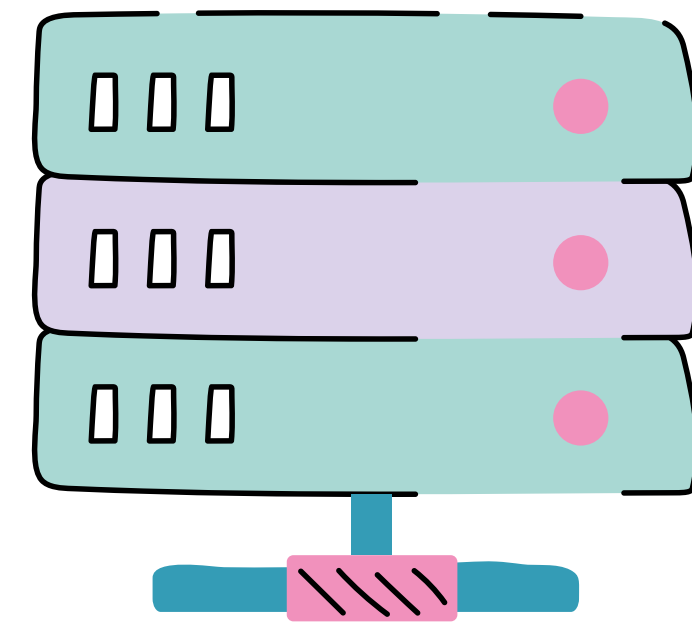
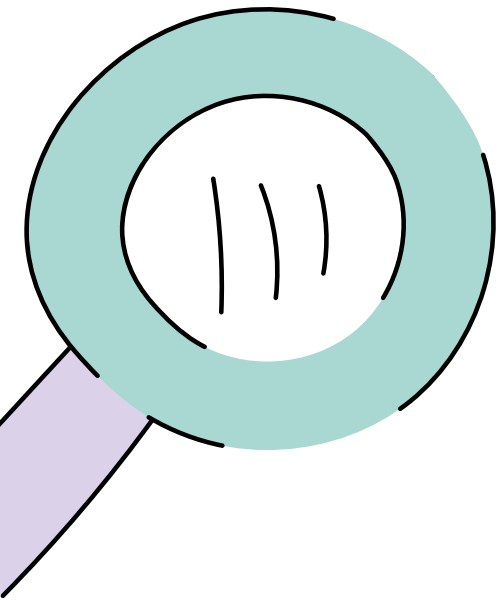
- O laço do-while significa faça enquanto, e é um comando utilizado para se realizar repetições com um diferencial que é a execução de pelo menos uma vez a instrução do laço, mesmo que a condição inicial seja falsa, pois a validação da condição só é feita depois da execução do laço.

```
#include <stdio.h>
// Função principal. Inicia a execução do programa
int main()
{
    int cont=0;
    do
    {
        printf("\n%d", cont);
        cont++;
    } while(cont<=10);
    return 0; // indica que o programa terminou com sucesso
}
```

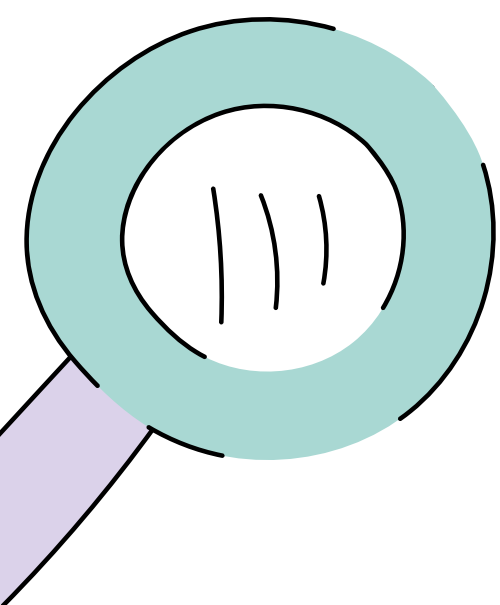


Arrays – Vetores e Matrizes

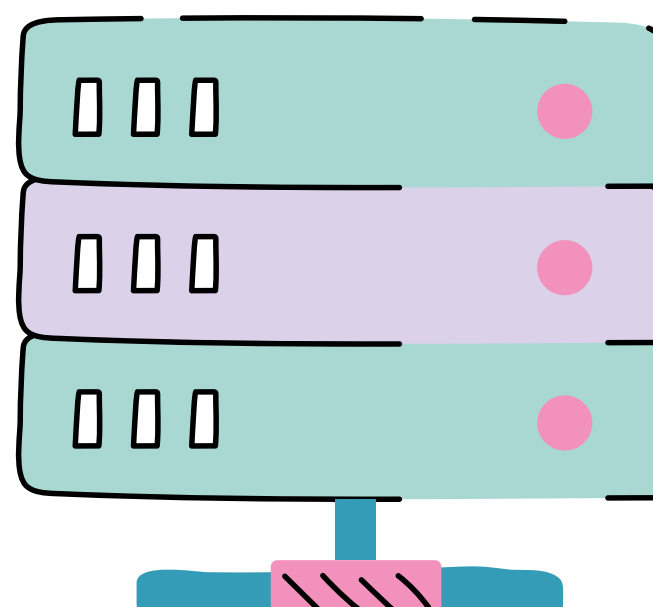
- Declaração
 - `int Vetor[5];` // declara um vetor de 5 posições
 - `int Matriz[5][3];` // declara uma matriz de 5 linhas e 3 colunas
- Acesso aos elementos do vetor
 - `Vetor[0] = 4;` // Coloca 4 na **primeira** posição de "Vetor"
 - `Vetor[4] = 8;` // Coloca 8 na **última** posição de "Vetor"



Arrays – Vetores e Matrizes

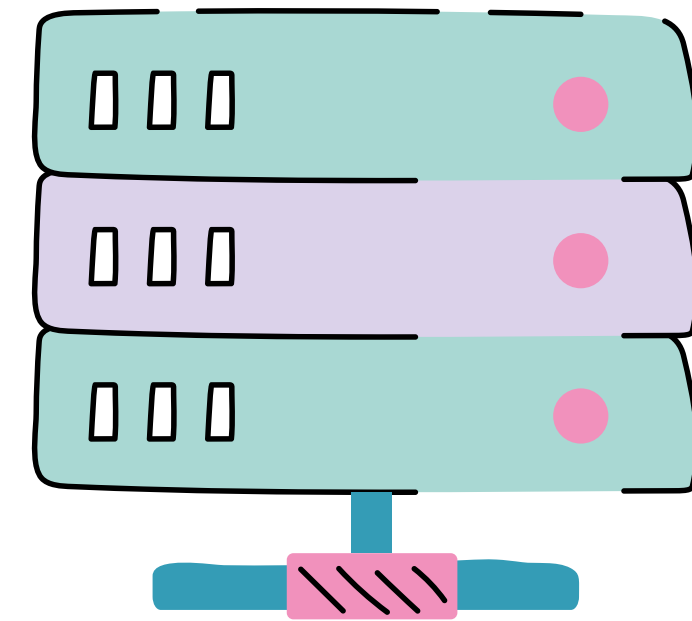
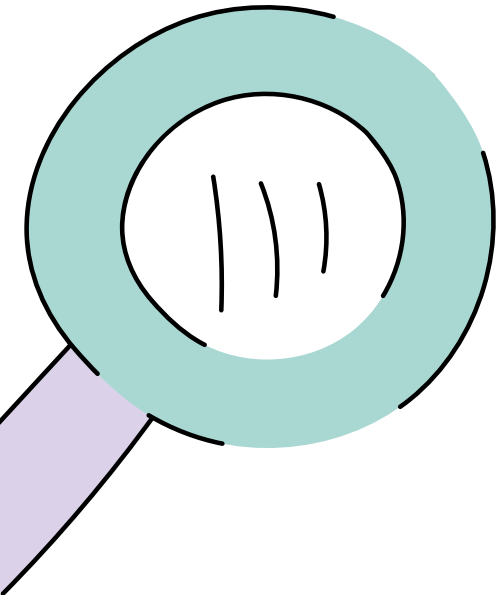


```
#include <stdio.h>
#include <stdlib.h>
// Função principal. Inicia a execução do programa
int main()
{
    int i;
    float nota[3], m=0;
    for(i=0;i<3;i++)
    {
        printf("\nDigite a nota %d", i+1);
        scanf("%f",&nota[i]);
        m = m+ nota[i];
    }
    m/=3;
    printf ("\nA media é %.2f\n",m);
    return 0; // indica que o programa terminou com sucesso
}
```



Arrays – Vetores e Matrizes

- Faça um programa que receba do usuário um vetor com 10 posições. Em seguida deverá ser impresso o maior e o menor elemento do vetor.
- Ler um conjunto de números inteiros, armazenando-o em vetor e calcular o quadrado das componentes deste vetor, armazenando o resultado em outro vetor. Os conjuntos têm 10 elementos cada. Imprimir todos os conjuntos.
- Faça um programa que lê três palavras do teclado e imprime as três palavras na ordem inversa que foram digitadas.
- Faça um programa que lê duas palavras do teclado e diz se elas são iguais ou diferentes. O programa deve dizer ainda se alguma das palavras digitadas é igual a “IFPI”.
- Desenvolva um programa para ler uma string (palavra) de no máximo 100 caracteres e informe o número que cada vogal se repete nessa frase.





ESTRUTURA DE DADOS

Dra. Maíla de Lima Claro
maila.claro@ifpi.edu.br

